

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

SE 423: Introduction to Mechatronics

Lecture 15: Coordinate Transforms & Dead-reckoning

Marius Juston

Wednesday, March 11th, 2026

Review – 5-minute partner discussion

Talk to the person next to you and answer these questions.

- Stand up and with your partner and:
  1. Agree on a starting position and a target position
  2. Discuss ways how one would go from point A to B as accurately as possible with their eyes closed.
  3. One of you closes your eyes (the partner should keep the other safe during the activity) and performs the strategy to try to get from point A to B. The other partner should not provide information during that time (Dead-Reckoning).
  4. Now redo the task but now your partner can only tell you distance to the target as well (type IR sensor).

Reminders

Spring Break next week, no office hours!

Lab: Starting Lab 5 this week. This a 1-week lab!

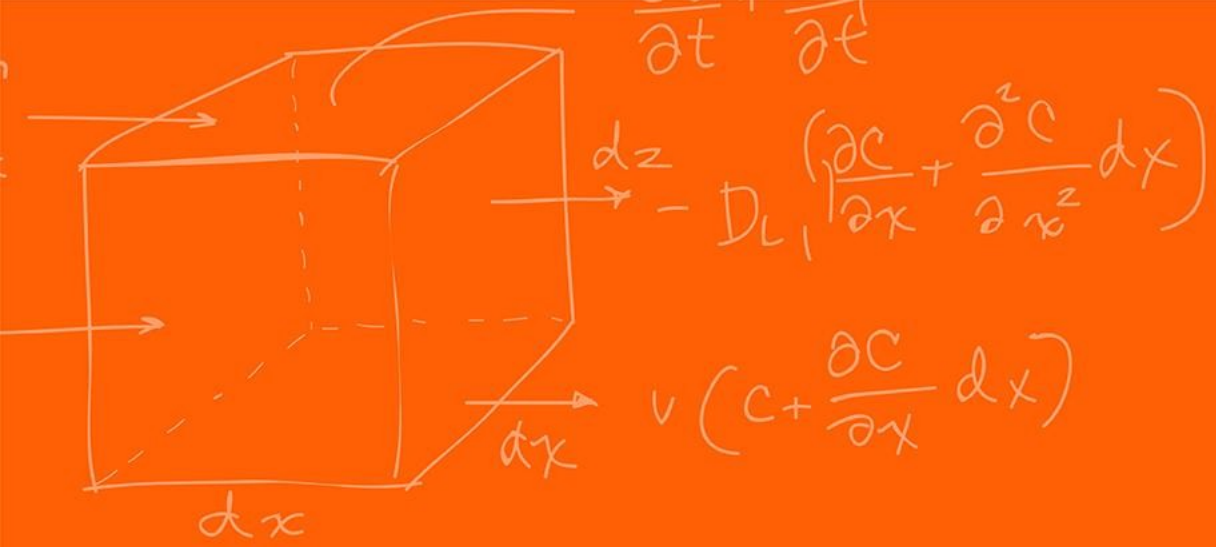
- Don’t forget the Lab 6 LabVIEW assignment that needs to be done beforehand!

Homeworks:

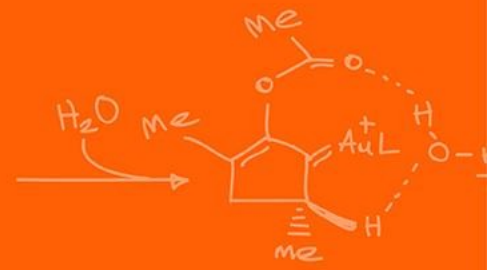
- Check-offs for homework 3 Ex 5 are due **March 24, 5PM (The end of office hours)**
  - This is a long homework, so **DO NOT** expect to work on and finish it during the Tuesday office hours time!
- Answers and code submissions for homework 3 are due **March 25, 9AM**
- Check-offs for homework 4 Ex 2 and 3 are due **March 31, 5PM (The end of office hours)**

Questions?

Homework, Lab, LabVIEW?



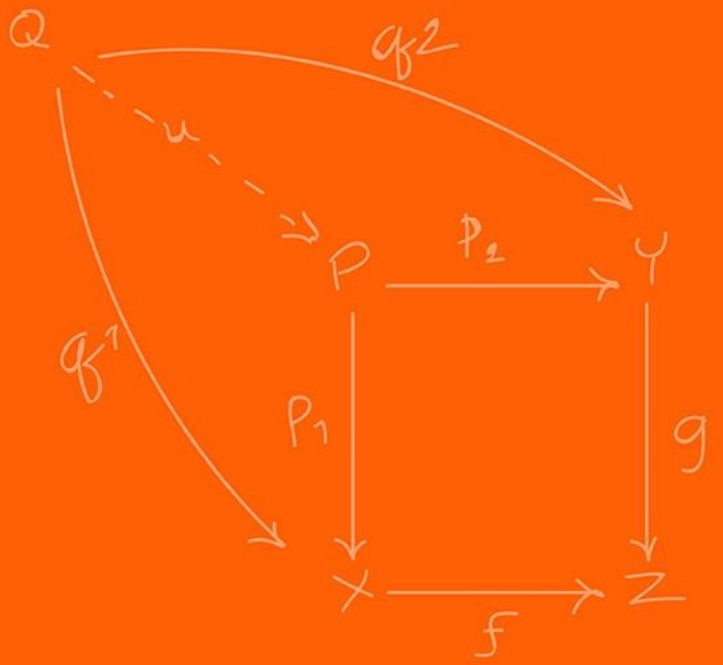




Localization

(A brief introduction.)





Localization

We now have sensors to figure out where obstacles are. We have sensors to figure out the internal state of the robot.

We also know how to make a robot drive straight or follow a wall using feedback control.

However, if we ask the robot to “go to coordinates (5, 5) on the map,” how does it know where it currently is?

This is the fundamental question of **Localization**!

Localization

To solve localization, we need to understand that the world is not perfect and not a single sensor gives the exact solution.

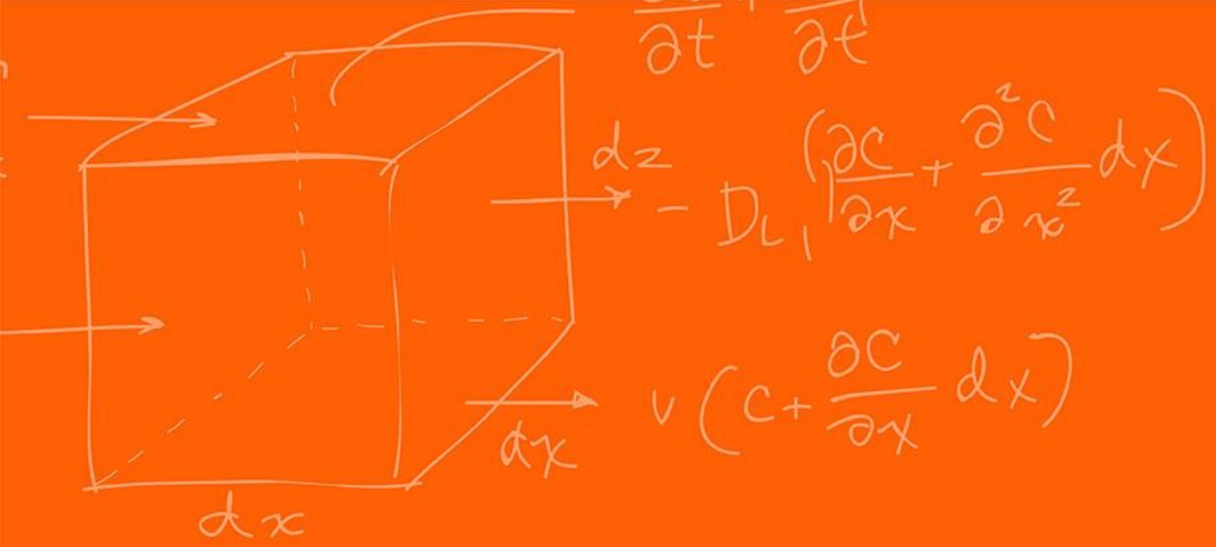
We first estimate movement using internal sensors (Dead-Reckoning).

What is a problem with just using internal sensors?

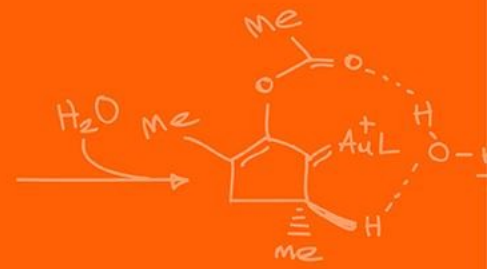
The internal sensors drift and accumulate errors!

Think of this as walking blind and relying on just counting your steps as you did at the beginning of the lecture.

Finally, we use external sensors (LiDAR, Cameras) to look at landmarks and correct these errors (SLAM)

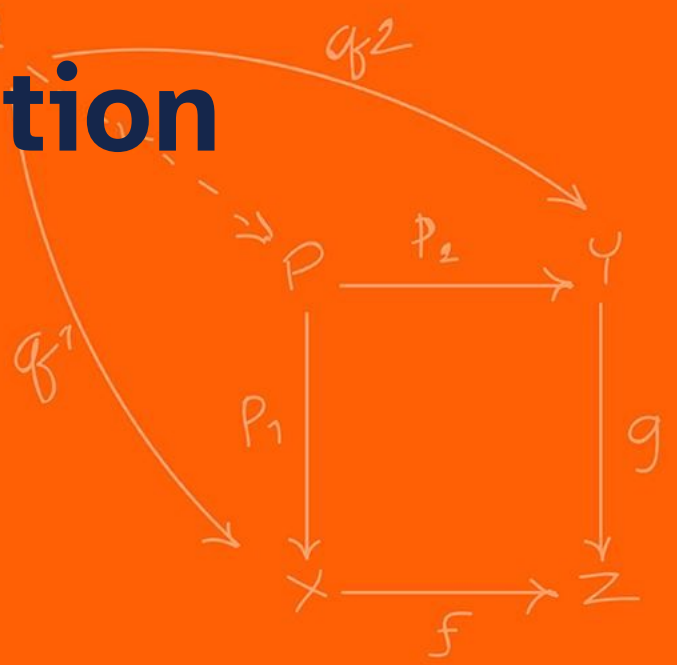








Coordinate Transformation



Localization

A robot experiences the world from its own perspective, which we call the **Local Frame** (or Body Frame).

The map or the room exists in a fixed, global reference frame, called the **Global / World Frame** (or Inertial Frame).

When driving a car, your local reference frame is your point of view inside the car, only you have that viewpoint, and the global reference is the shared viewpoint, which is provided by the GPS map.

Each of the sensors view its information in its own Local Frame; however, to navigate a room, we must translate the local sensor data into the global map!

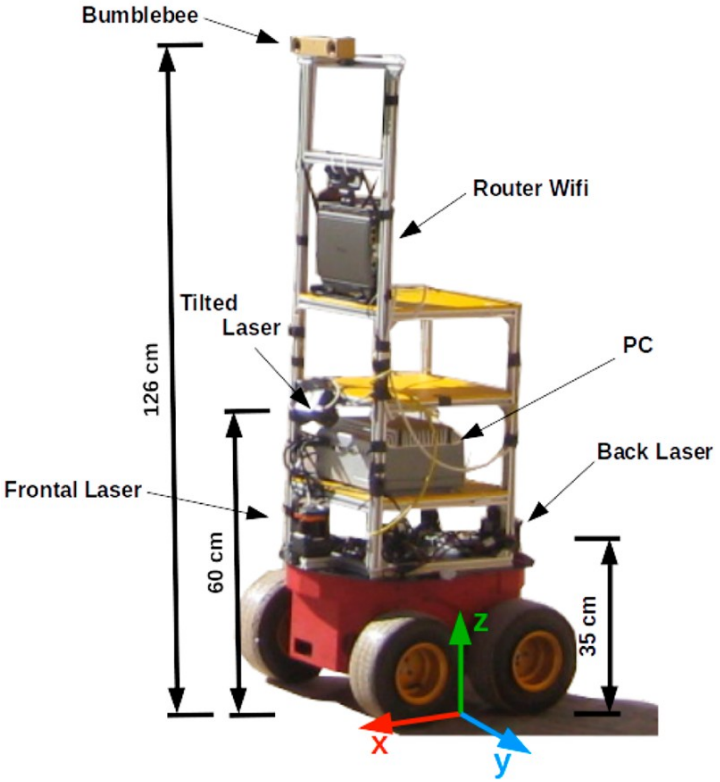
Localization

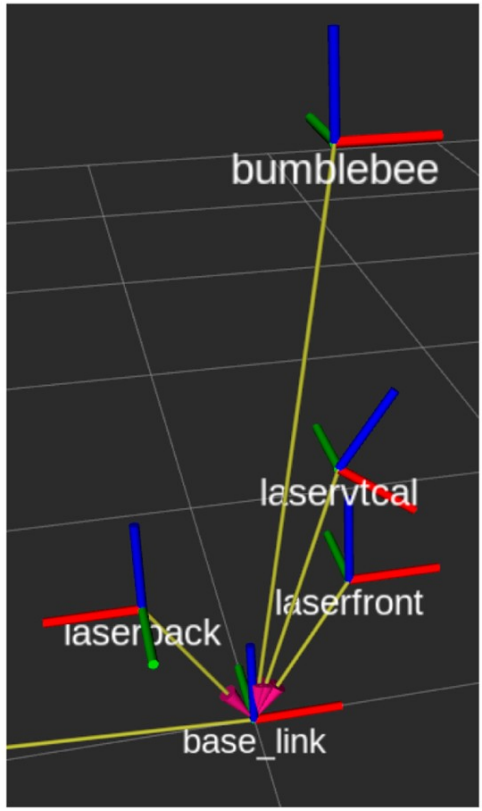
The local frames of each of the sensors can be of different positions and orientation than the robot's **body frame**.

The **body frame** is typically the frame where you define the (x, y, θ) of the robot.

So, we need to translate the sensor frames to the body frame, and then the body frame to the global frame.

If the robot to the right moves "forward 1 meter", does its local x coordinate increase or it's y? What about it's world x, y coordinates?





https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1671673/full

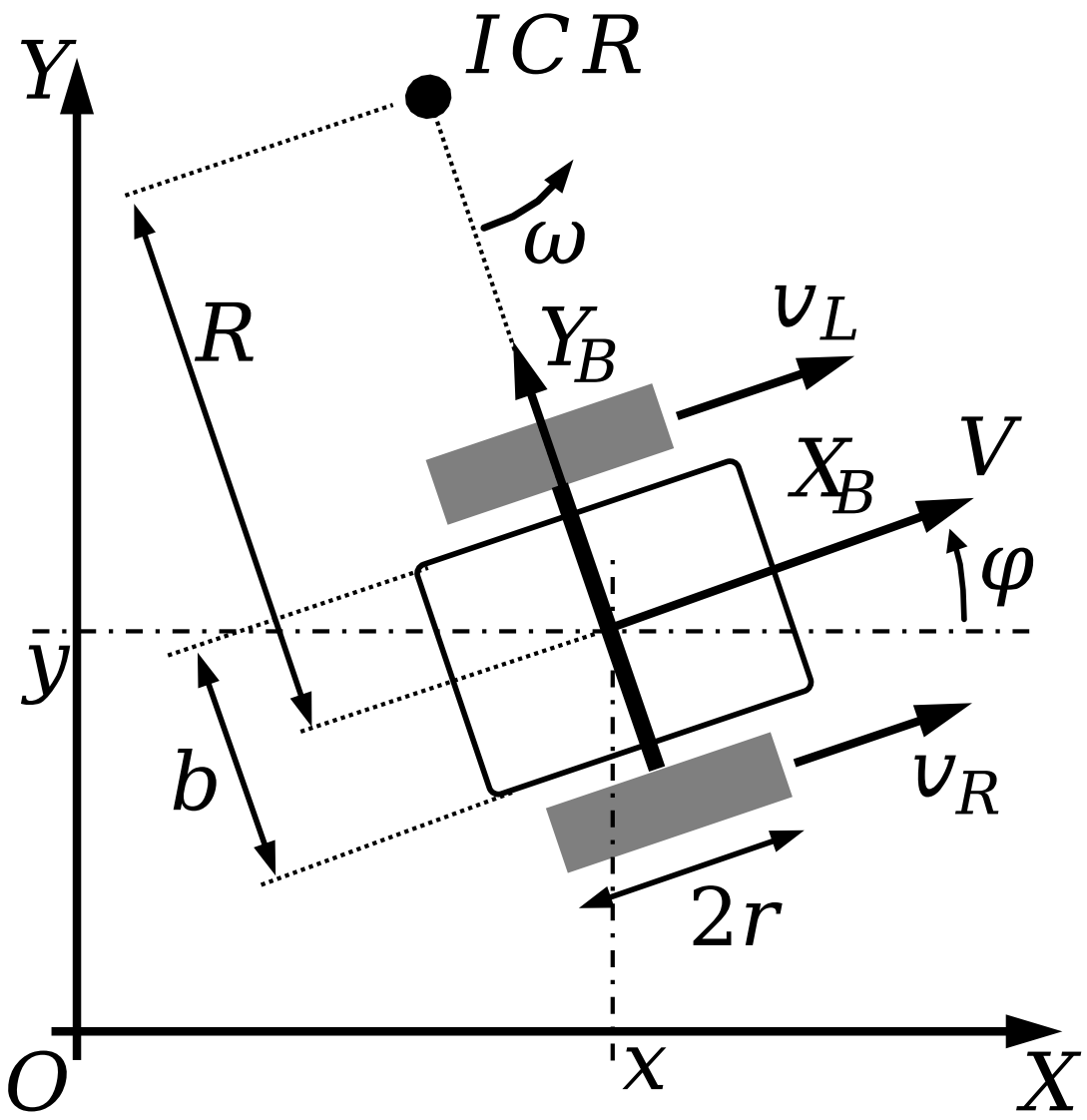
Localization

Let the **World Frame** be defined by axes X, Y with origin O.

Let the **Body Frame** be defined by axes X_B (pointing forward) and Y_B (pointing left) with origin O_B.

The robot's pose in the world is defined by its position and it's heading angle.

$$p = (x, y, \theta)$$



https://en.wikipedia.org/wiki/Differential_wheeled_robot

Localization – 2D Translation (Shifting)

- If the robot's heading $\theta = 0$, its axes are perfectly aligned with the world axes.

Thus, moving from the robot frame to the world frame is simply a vector addition!

If the robot is at world coordinates $\mathrm{T} = (x, y)$, a point $P_R = (x_r, y_r)$ in the robot frame is:

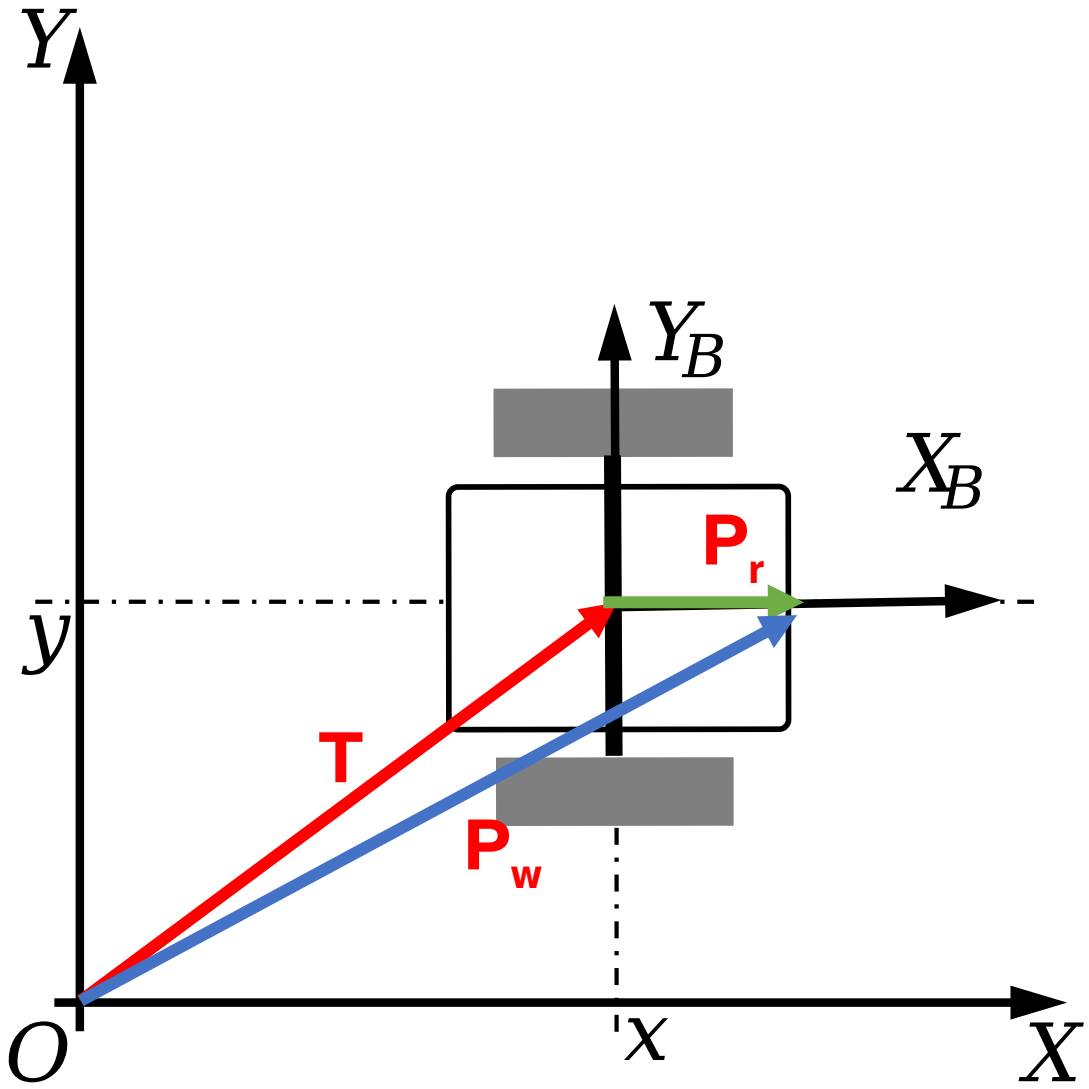
$$P_w = (x_r + x, y_r + y)$$

In the world frame. In vector math, this is represented as:

$$P_W = P_R + T$$

Translation is linear and simple; however, robot also rotate.

Localization – 2D Translation (Shifting)



Localization – 2D Rotation

- Suppose the robot frame is at the world origin but rotated by and angle φ.

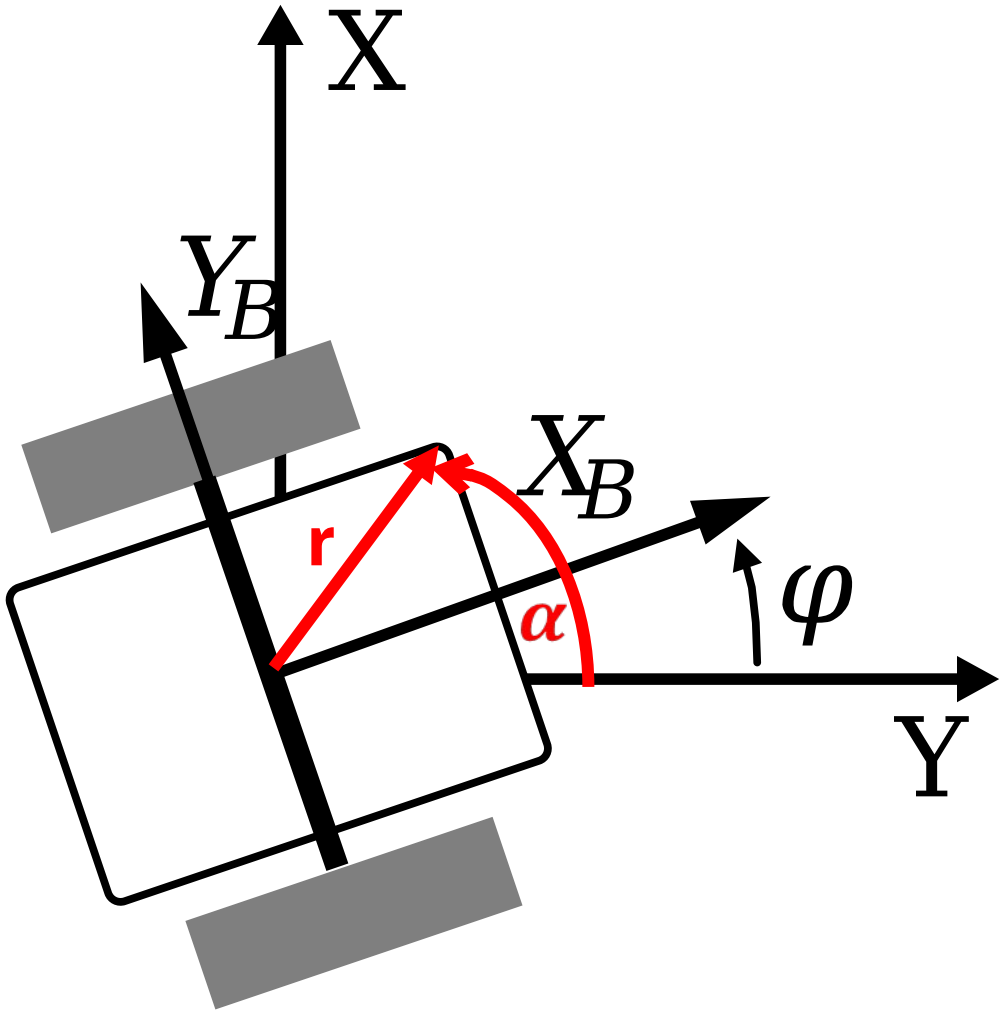
We want to map point P, from the rotated frame to the fixed frame.

Let point P have polar coordinates (r, α)

Therefore,

$$(x_r, y_r) = (r\cos(\alpha), r\sin(\alpha))$$

Localization – 2D Translation (Shifting)



Localization – 2D Rotation

In the world frame, the angle of point P is now at $(\alpha + \varphi)$.

The world coordinates are thus,

$$P_w = (r\cos(\alpha + \varphi), r\sin(\alpha + \varphi))$$

We can expand the following using standard trigonometric sum identities:

$$x_W = r(\cos(\alpha)\cos(\varphi) - \sin(\alpha)\sin(\varphi))$$
$$y_W = r(\sin(\alpha)\cos(\varphi) + \cos(\alpha)\sin(\varphi))$$

Localization – 2D Rotation

Substituting back in the location position:

$$(x_r, y_r) = (r\cos(\alpha), r\sin(\alpha))$$

To the global position:

$$x_W = r(\cos(\alpha)\cos(\varphi) - \sin(\alpha)\sin(\varphi))$$
$$y_W = r(\sin(\alpha)\cos(\varphi) + \cos(\alpha)\sin(\varphi))$$

We thus get:

$$x_W = x_R\cos(\varphi) - y_R\sin(\varphi)$$
$$y_W = x_R\sin(\varphi) + y_R\cos(\varphi)$$

Which is elegantly expressed using matrix multiplications!

Localization – 2D Rotation

The system of equations can be written as a 2x2 matrix $\mathrm{P_w} = R(\varphi)\mathrm{P_r}$

$$\begin{bmatrix} x_W \\ y_W \end{bmatrix} = \begin{bmatrix} \cos(\varphi) & -\sin(\varphi) \\ \sin(\varphi) & \cos(\varphi) \end{bmatrix} \begin{bmatrix} x_R \\ y_R \end{bmatrix}$$

The matrix $\mathrm{R}(\varphi)$, belongs to the Special Orthogonal Group SO(2).

Given that the determinant represents how much the space gets scaled after the transformation. What is the determinant of rotation matrices without doing any calculation?

It should be 1, meaning it preserves the scale and area! The robot doesn't stretch as it turns!

Localization – Transformation

- In reality, a robot is both translated and rotated!

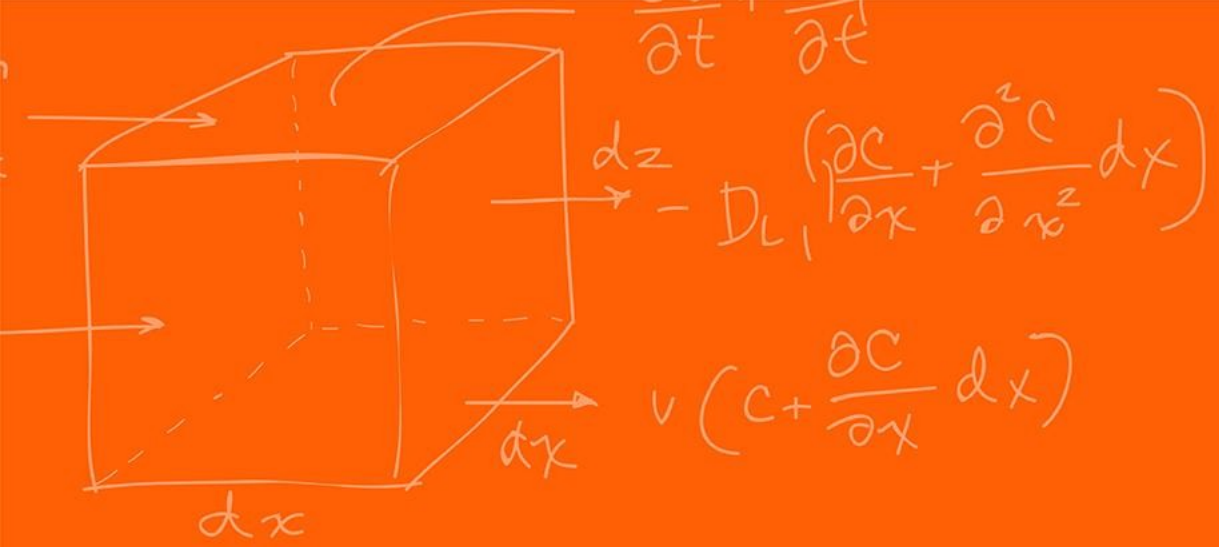
Thus, the complete transformation is:

$$P_w = R(\varphi)P_r + T$$

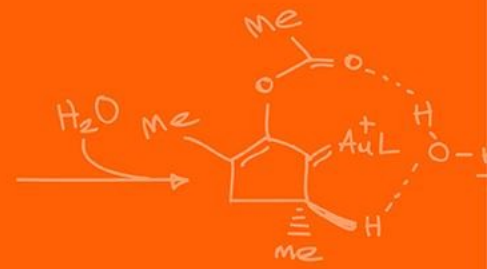
We can, however, make this into a single matrix operation, through Homogeneous Coordinates,

$$\begin{bmatrix} x_W \\ y_W \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_R \\ y_R \\ 1 \end{bmatrix}$$

This is the standard representation used in all modern software!

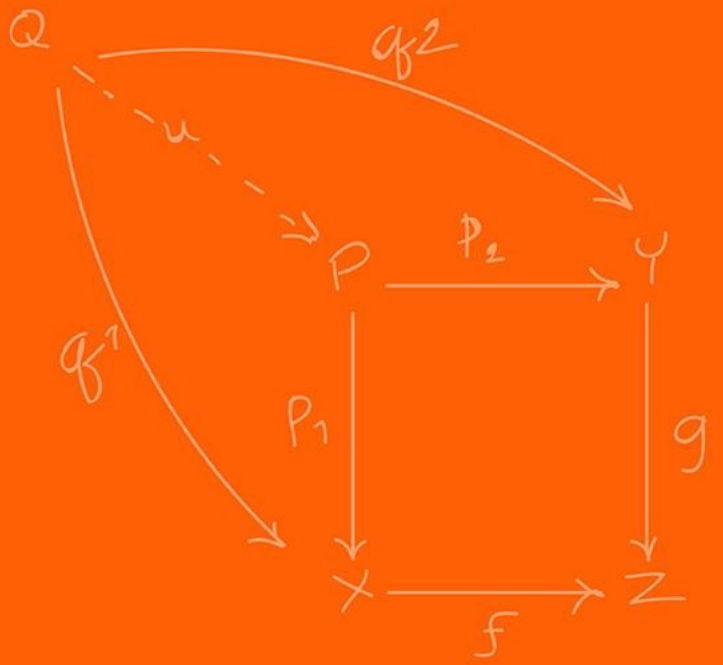








Dead-Reckoning



Dead-Reckoning

Now that we can map local coordinates to the world, how do we track our movement?

Dead-Reckoning (Odometry) is the process of calculating current position by integrating previously determined positions and estimated speeds over time.

It relies purely on internal sensors, primarily wheel encoders and IMUs.

Dead-Reckoning

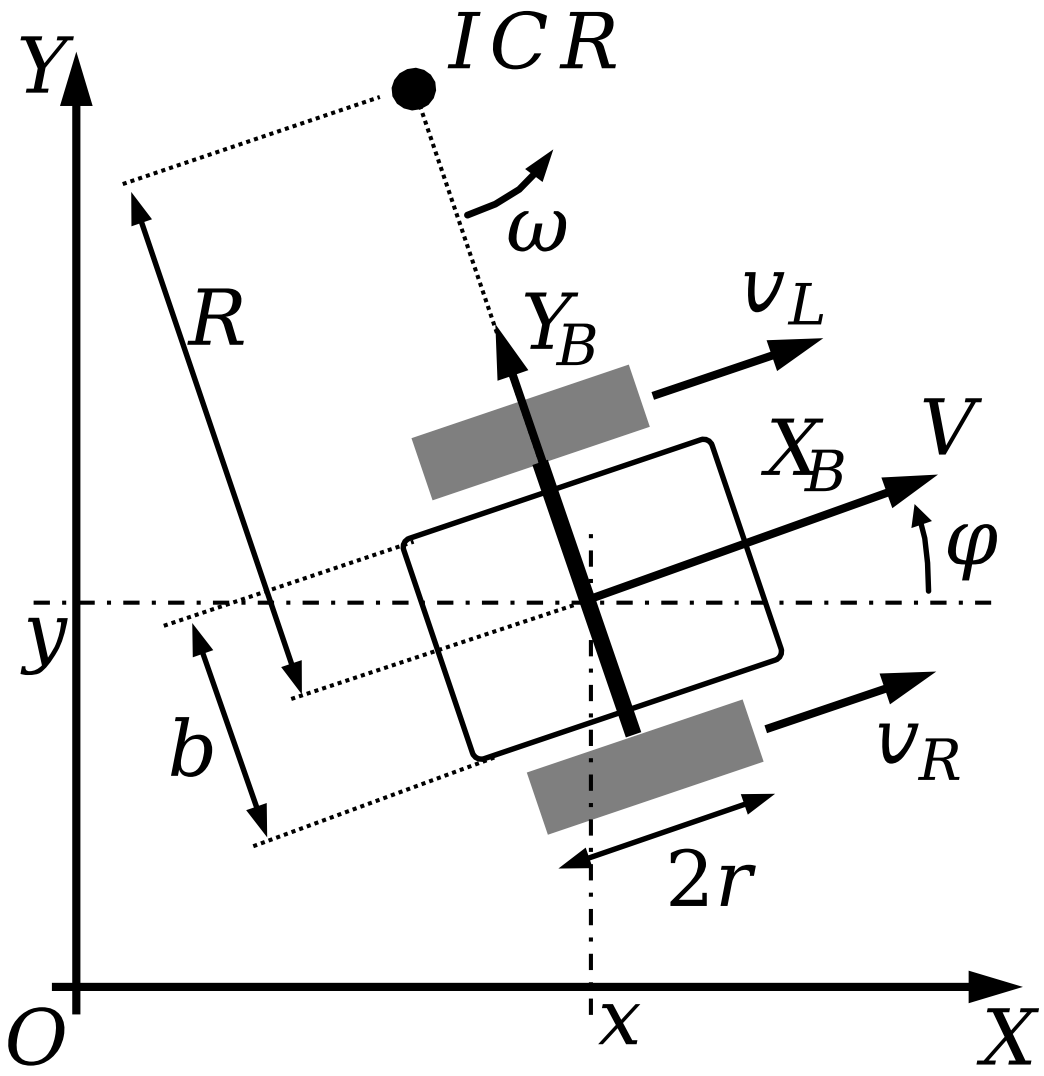
Our robot uses a differential drive system (2 independently driven wheels).

The robot's overall forward linear velocity V is the average of the two wheels:

$$V = \frac{v_L + v_R}{2}$$

If $v_L = v_R$, what is the robot doing?

The robot goes forward!



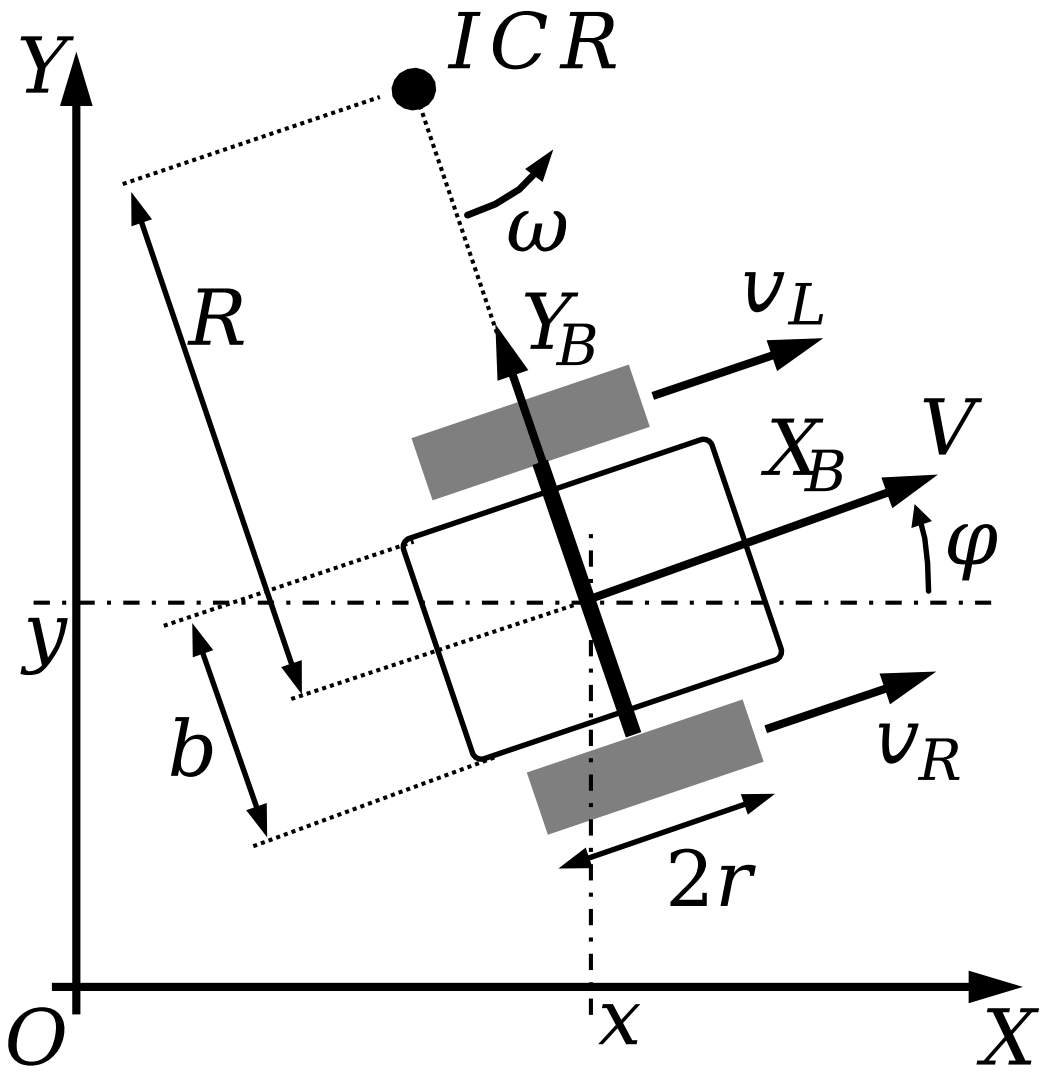
Dead-Reckoning

The robot's angular velocity ω depends on the difference between the wheel speeds.

$$\omega = \frac{v_R - v_L}{b}$$

If $v_L = -v_R$, what is the robot doing?

The robot rotates in place (a zero-radius turn).



Dead-Reckoning

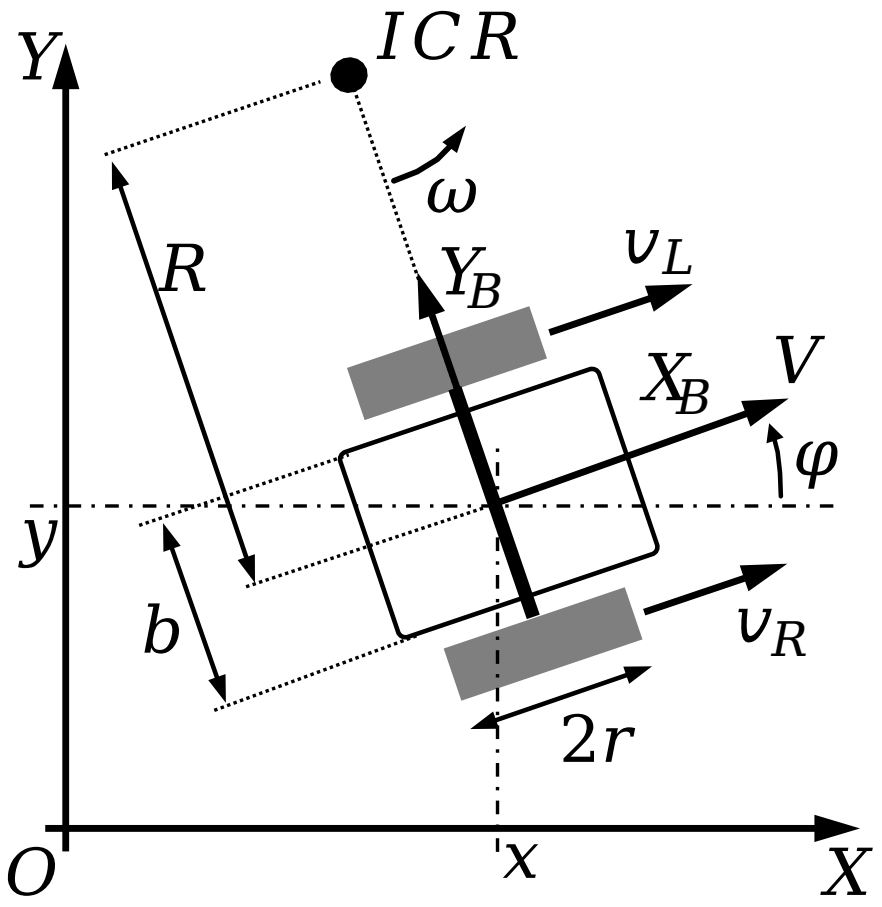
Assuming that the wheels are at contacting the ground, the wheels describe arcs in the plane in such a way that the robot always rotates around a point (the **ICR** refers to the instantaneous center of rotation).

The rotation leads the robot to have the rotation of the vehicle to be ω around the **ICR**.

Following the angular velocity equation $\omega = \mathrm{r}\nu$, we have,

$$\omega = \left(R + \frac{\mathrm{b}}{2}\right)\nu_{\mathrm{r}}$$

$$\omega = \left(R - \frac{\mathrm{b}}{2}\right)\nu_{\mathrm{l}}$$



Dead-Reckoning

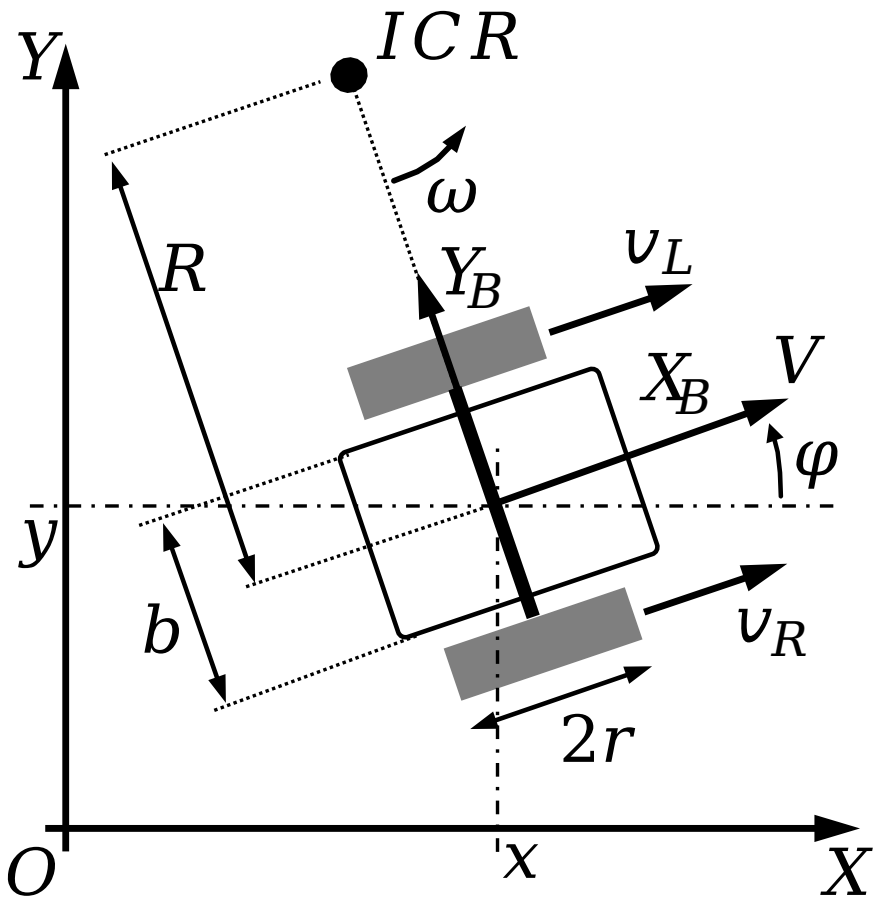
Solving for ω and R, we get that,

$$\omega = \frac{v_R - v_L}{b}$$

$$R = \frac{b}{2}\frac{v_R + v_L}{v_R - v_L}$$

Very simple!

When $R = \infty$, what is the robot doing?
When $R = 0$, what is the robot doing?



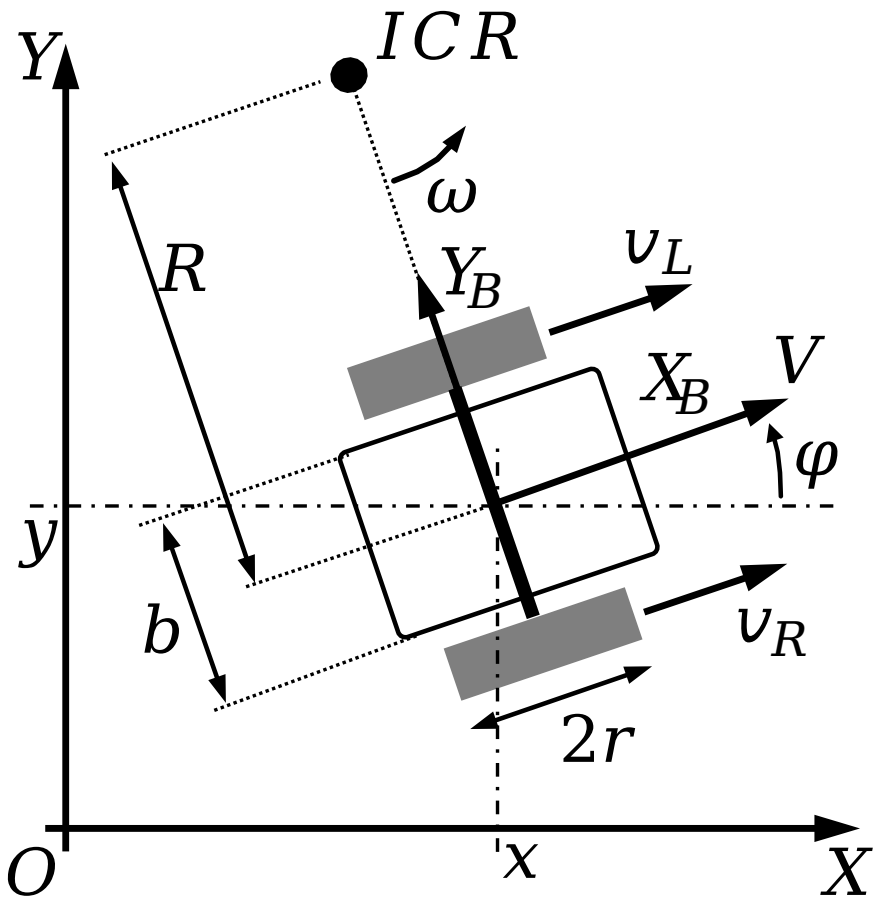
Dead-Reckoning

Based on the calculated forward velocity from the encoders, V, in lab we already calculate ν_R, ν_L.

Using forward Euler integration, we update our world X and Y coordinates using our current heading φ_t.

$$x_{t+1} = x_t + V_t \cos(\varphi_t)\,\Delta t$$
$$y_{t+1} = y_t + V_t \sin(\varphi_t)\,\Delta t$$
$$\varphi_{t+1} = \varphi_t + \omega_t \Delta t$$

There we go!
We now have 2 ways to get the robot's position, integrating the **IMU** or integrating the **wheel encoder** information!

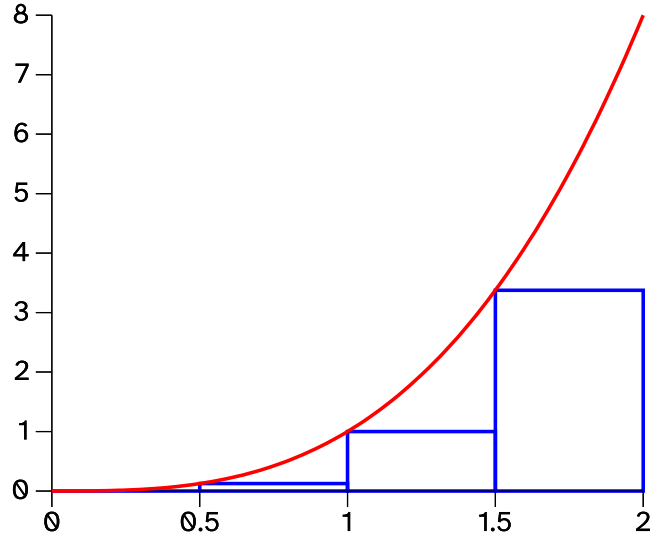


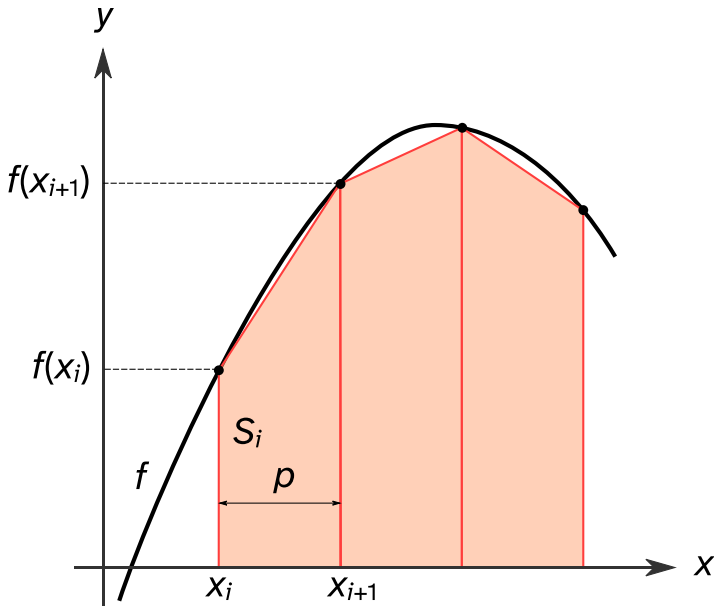
Dead-Reckoning

There are multiple ways of integrating; however, we can get a slightly more accurate result depending on how we do it.

Specifically, instead of doing the Riemann sum (rectangle rule) we can do the Trapezoidal rule. In lab x, represents the heading φ, and $f(\cdot)$, represents the gyro reading.

$$x_t = x_{t-1} + \frac{1}{2}(f(x_t) + f(x_{t-1}))\Delta t$$





Dead-Reckoning

Given the 2 methods of calculating the robot's position, if we run this integration for 10 minutes, will the robot's calculated position match its real physical location?

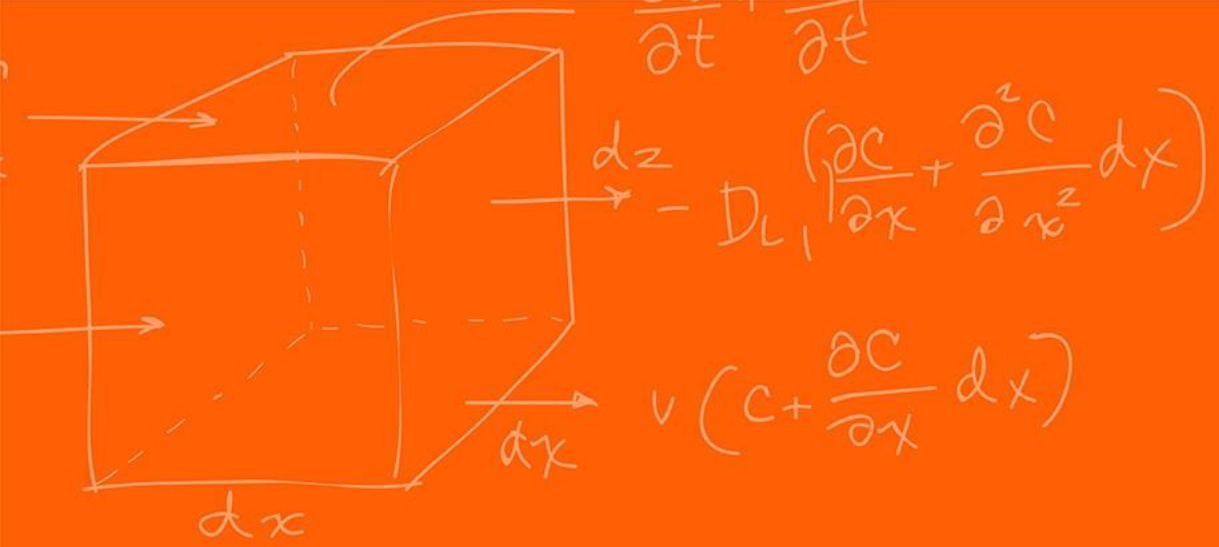
Will the IMU and the wheel encoder measurements match?

Absolutely not! Dead-reckoning is subject to unbounded accumulating errors.

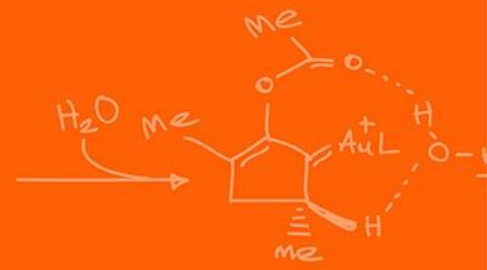
Any small measurement error is added to the previous state, meaning the error grows infinitely over time.

This is known as drift. **Odometry Drift** for wheel encoders and **IMU Drift** for the IMU.

In later lectures we will learn about the **Kalman Filter** to be able to **fuse** both sensors to get a more accurate measurement!

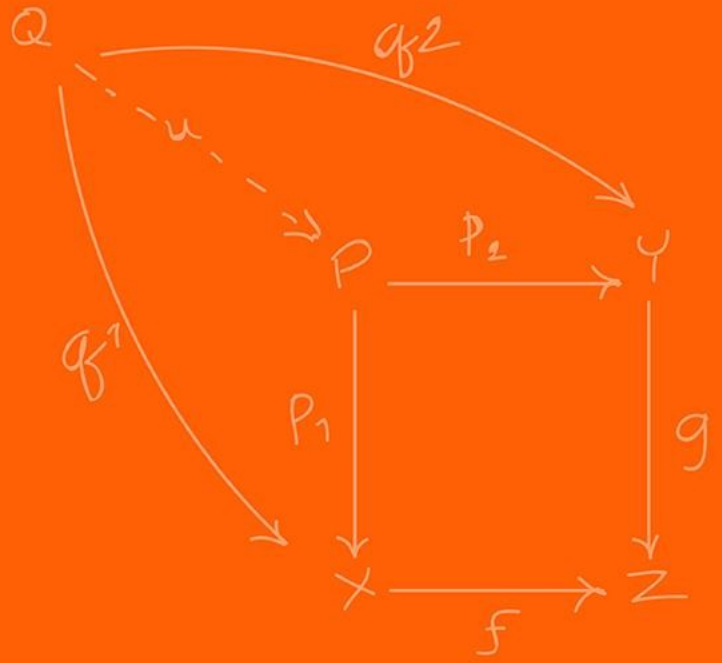






Errors





Errors

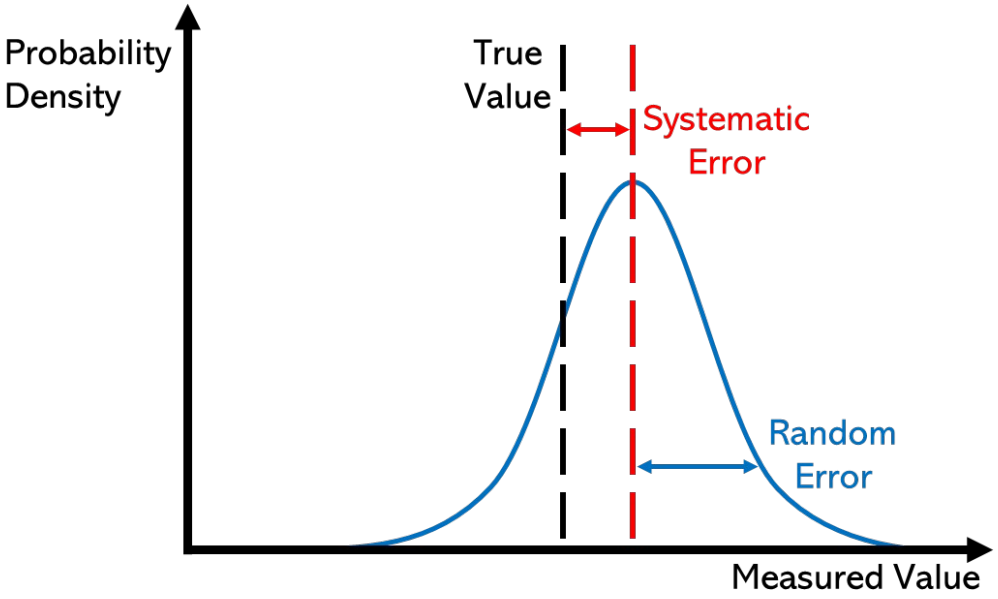
There are 2-types of errors in measurements,

Systematic Error:

- Built into the physical mechanism of the sensor
- Always occurs, with the same value, when we the instrument in the same way and in the same case
- Can often be accounted for

Examples:

- Unequal wheel diameter
- Constant accelerometer bias



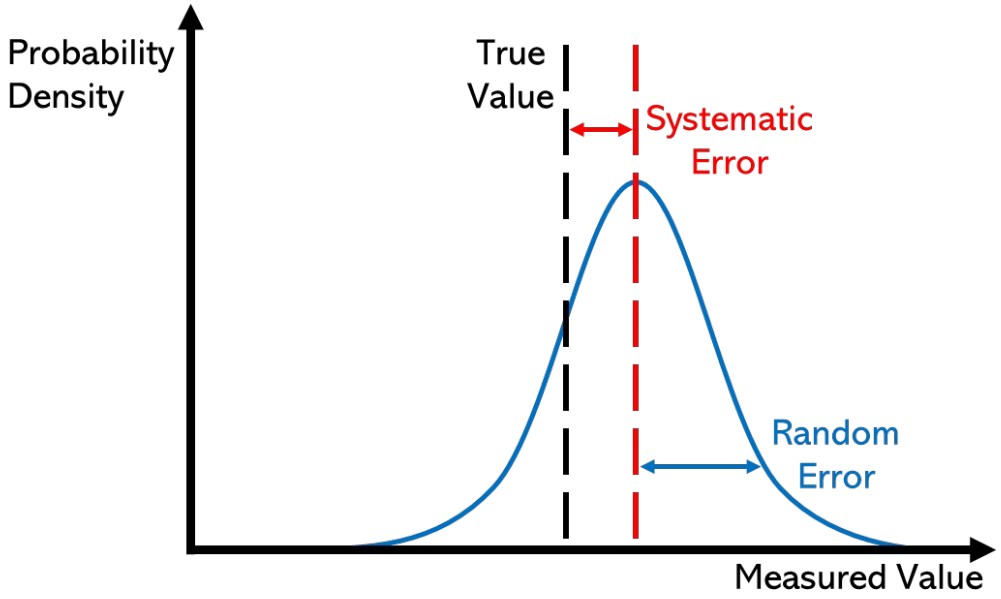
https://en.wikipedia.org/wiki/Observational_error

Errors

There are 2-types of errors in measurements,

Random Error:

- Random and due to environment factors
- May vary from observation to another
- These fluctuations may be in part due to interference of the environment with the measurement process
- Much harder to model (often assumed to be Gaussian)



https://en.wikipedia.org/wiki/Observational_error

Examples:

- Wheel slip
- Loose components

Errors

In robotics the best way to handle noise is just by adding more sensors / different sensors.

The more different opinions / viewpoints you have the more likely that the majority will agree or will provide useful information to help compensate from the fault of others!

How does one calculate how much the error drift?

To help combat heading drift from encoders, we use the Z-axis rate gyro from our IMU.

$$\varphi_{\text{gyro},t+1} = \varphi_{\text{gyro},t} + \omega_{\text{gyro},t}\Delta t$$

Errors

A Gyroscope's output is:

$$\omega_{\text{gyro},t} = \omega_{\text{true},t} + \epsilon_{\text{bias}} + \epsilon_{\text{noise}}$$

We assume that the noise $\epsilon_{\text{noise}} \sim \mathcal{N}(0, \sigma^2)$, is "White Noise", zero mean Gaussian noise.

When we integrate the bias over time we:

$$\int \epsilon_{\text{bias}}\, dt = \epsilon_{\text{bias}} \Delta t$$

If the bias just 0.05 degrees / sec, how far off is our heading after 5 minutes?

15 degrees!

This is why we calibrate the IMU to measure and subtract the bias!

Errors

Now with the bias removed we now have White Noise that remains.

$$\omega_{\text{gyro},t} = \omega_{\text{true},t} + \epsilon_{\text{noise}}$$

Integrating white noise results in a "**Random Walk**" (a Wiener process in stochastic calculus).

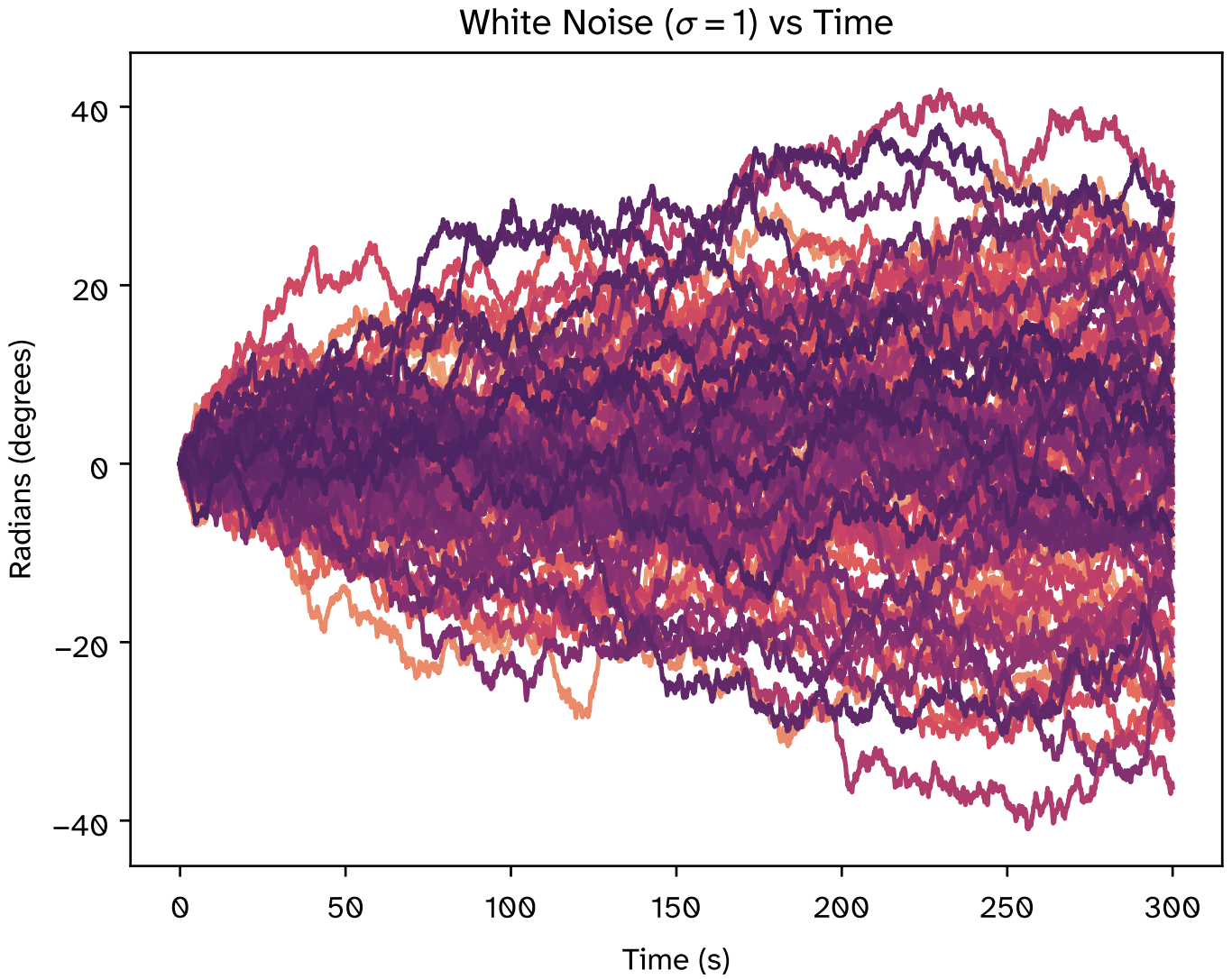
The mean of the error over multiple "Random Walk" runs is centered at 0.

The variance of the error grows linearly with time $\sigma_{\varphi}^2 = \sigma_{\omega}^2 \Delta t$.

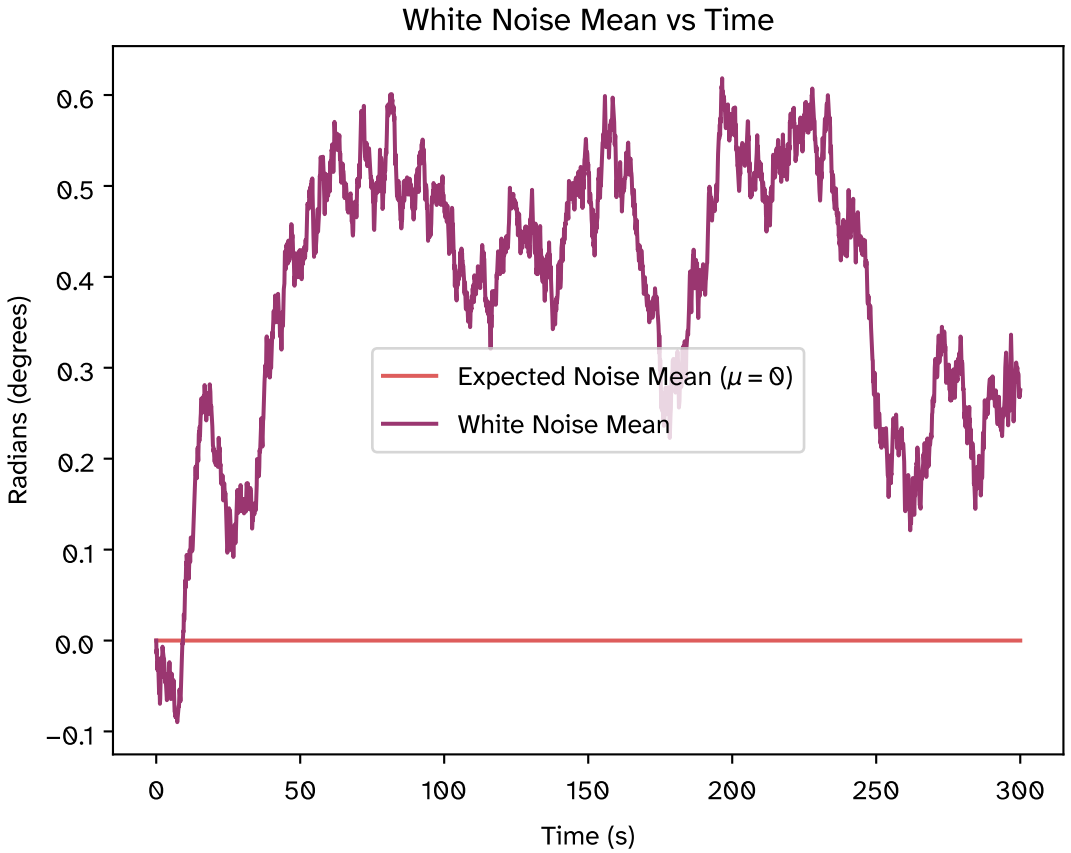
The standard deviation (the actual angular error) grows proportional to the square root of time $\propto \sqrt{t}$.
The math for proving this is very complicated, so just take it as truth!

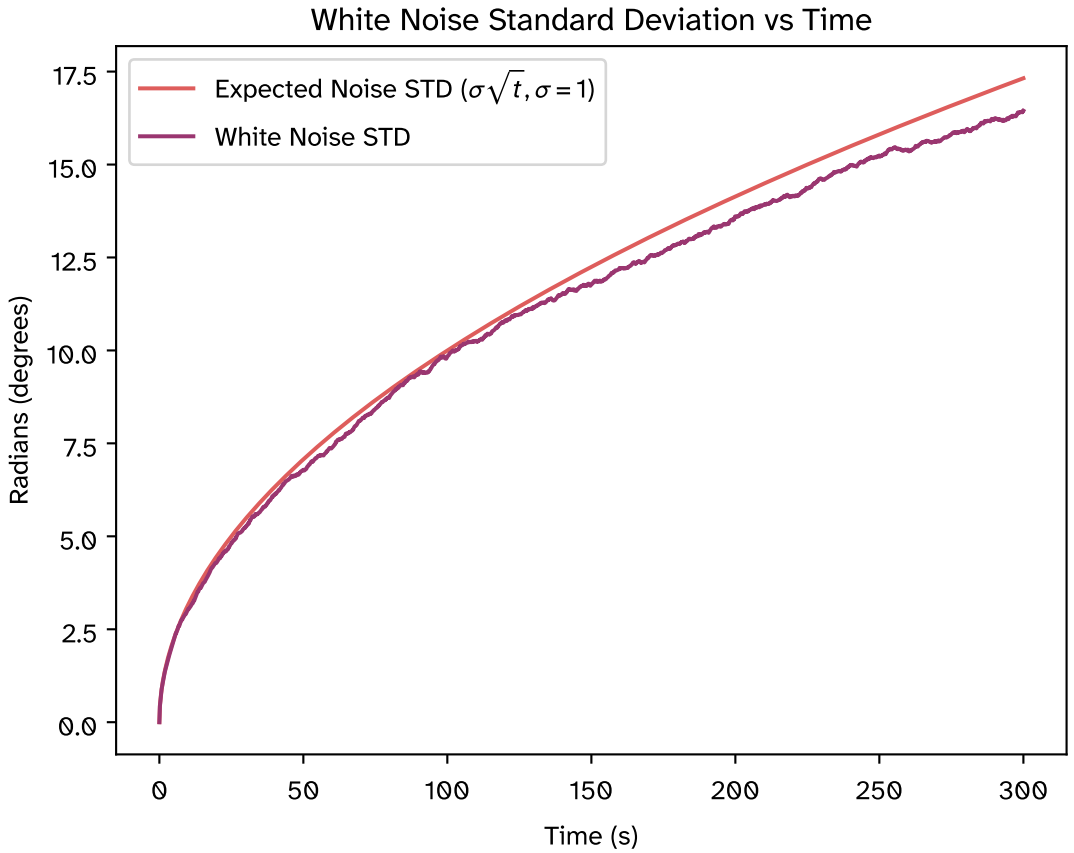
Thus, over long period, the gyro's heading becomes entirely unreliable!

Errors


White Noise ($\sigma = 1$) vs Time
Radians (degrees)
40
20
0
−20
−40
0
50
100
150
200
250
300
Time (s)

Errors


White Noise Mean vs Time
Radians (degrees)
Time (s)
Expected Noise Mean ($\mu = 0$)
White Noise Mean
0.6
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
0
50
100
150
200
250
300


White Noise Standard Deviation vs Time
Radians (degrees)
Time (s)
Expected Noise STD ($\sigma\sqrt{t}, \sigma = 1$)
White Noise STD
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
0
50
100
150
200
250
300

Errors

Encoders are good at low speeds and measuring exact distance, but terrible at heading due to wheel slip.

Gyroscopes are highly accurate for rapid rotations and **immune to wheel slip**, but drift over time due to integrated noise.

Sensor Fusion is the mathematical process of combining these complementary sensors.

We use algorithms like Complementary Filters or Kalman Filters to merge the data.

Errors

Very briefly the Complementary Filter (very simple filter) blends a **high-pass** filtered signal and a **low-pass** filtered signal.

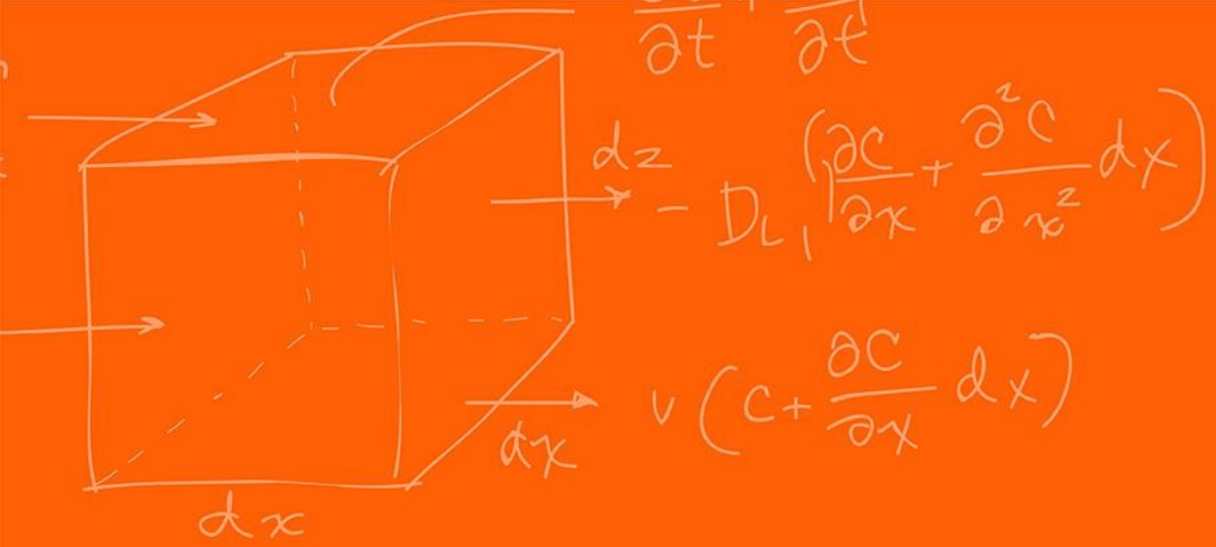
For heading, we pass the integrated Gyro data through a **High-Pass** filter (trusting it in the short term).

We pass an absolute heading measurement (like a Magnetometer, if available) through a **Low-Pass** filter (trusting it in the long term).

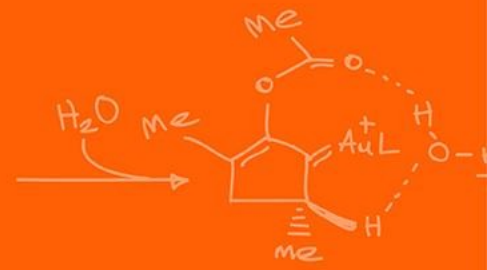
$$\varphi_{fused} = \alpha\left(\varphi_{fused} + \omega_{gyro}\Delta t\right) + (1-\alpha)\left(\varphi_{mag}\right)$$

Where α is a tuning constant close to 1 (e.g., 0.98).

This is essential an **exponential moving average**,



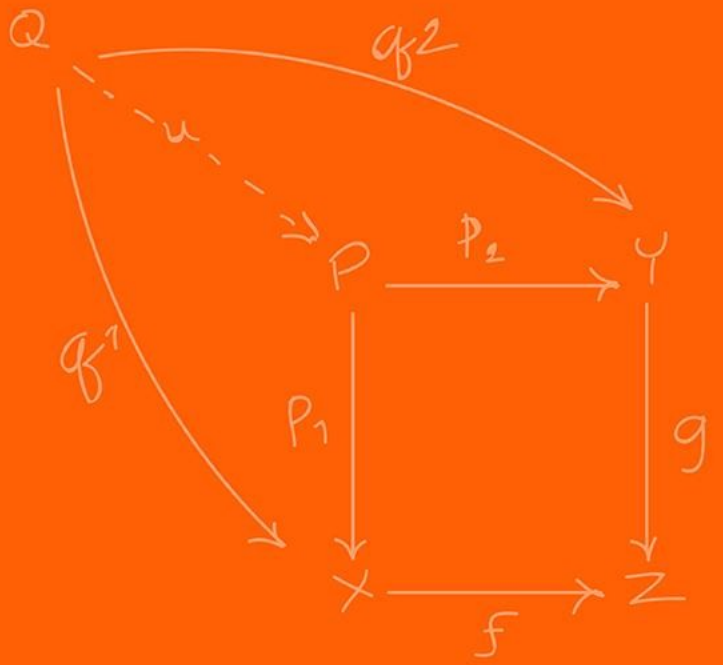






SLAM

(A VERY brief introduction.)



SLAM

Even with perfect sensor fusion of internal sensors, drift will eventually occur.

To truly fix our global position, we must look at the external world using active or passive sensors.

We detect **Landmarks**: recognizable, distinct features in the environment with known coordinates.

When we see a landmark, we calculate our relative distance to it and use that to instantly reset our accumulated odometry error.

When you are driving / walking and you are lost what are some landmarks that you use to relocate yourself?

Examples:

- Specific buildings
- Specific road names

SLAM

The 2 sensors that we can use in our robot is the LiDAR and the camera.

For the camera, we can create specific tags to place in the environment as landmarks. The most common is called AprilTags,

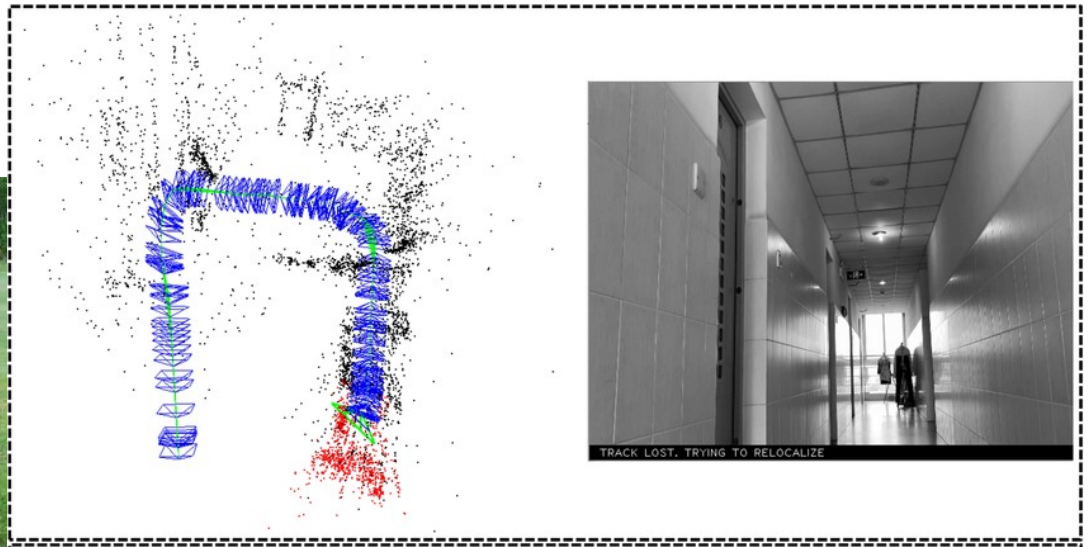
If you cannot place landmarks in your environment, then it becomes much more complicated as a process:

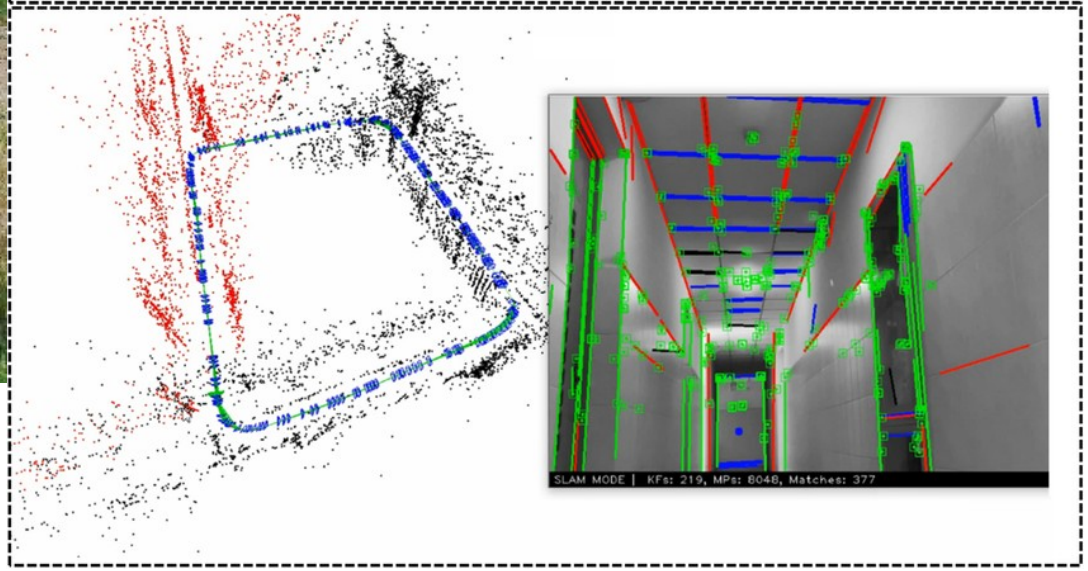
- Look at images, find pixels that represent landmarks
- At the next frame match to find those previous landmarks
- Calculate the 6DOF movement your camera did to achieve the landmark transform
- Calculculate the odometry
- If you perform a loop, correct any error through loop closure

SLAM



https://april.eecs.umich.edu/software/apriltag





https://www.researchgate.net/publication/355183350_PSL-SLAM_a_monocular_SLAM_system_using_points_and_structure_lines_in_Manhattan_World

SLAM

What if there are no AprilTags and we don’t know the map beforehand?

We use **SLAM: Simultaneous Localization and Mapping**.

The rapidly spinning LiDAR generates a dense 2D point cloud.

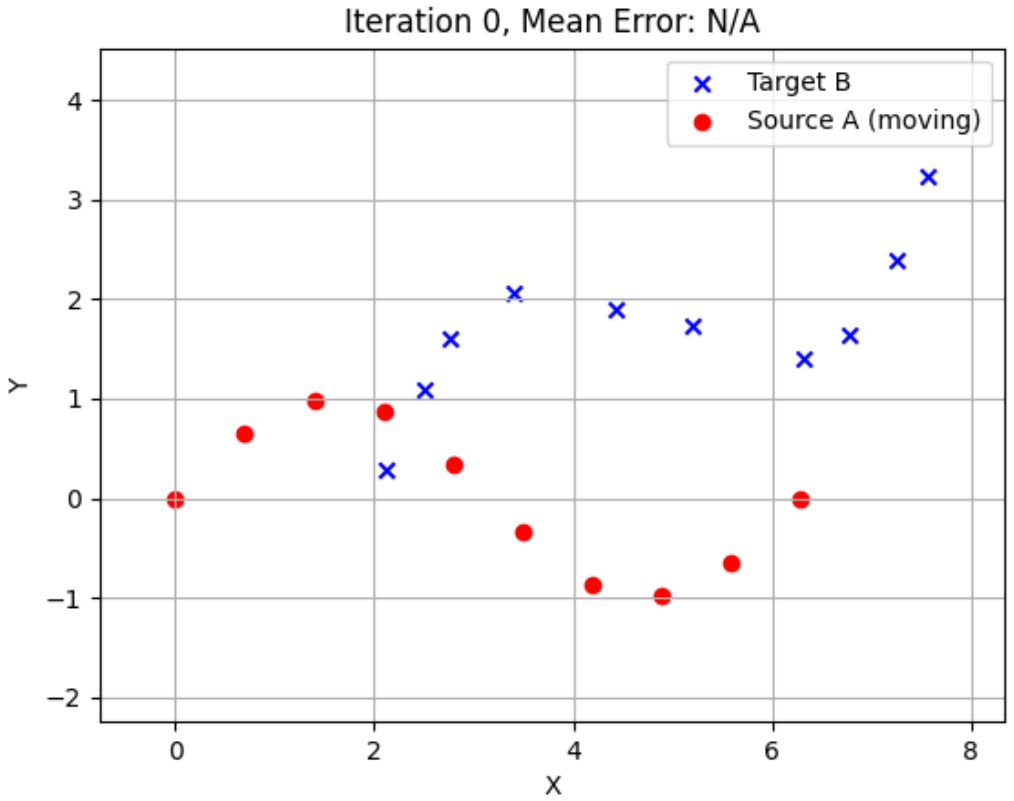
As the robot moves, it takes a new point cloud scan and compares it to the previous scan.

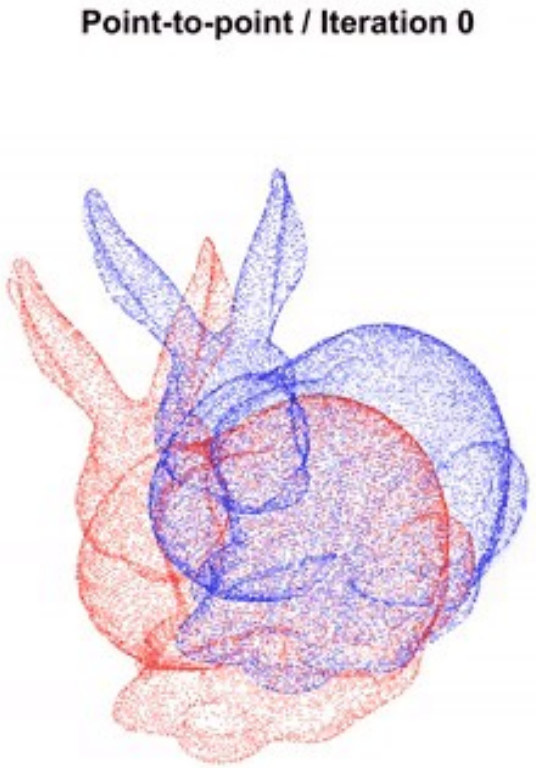
How do we align two-point clouds taken a few seconds apart?

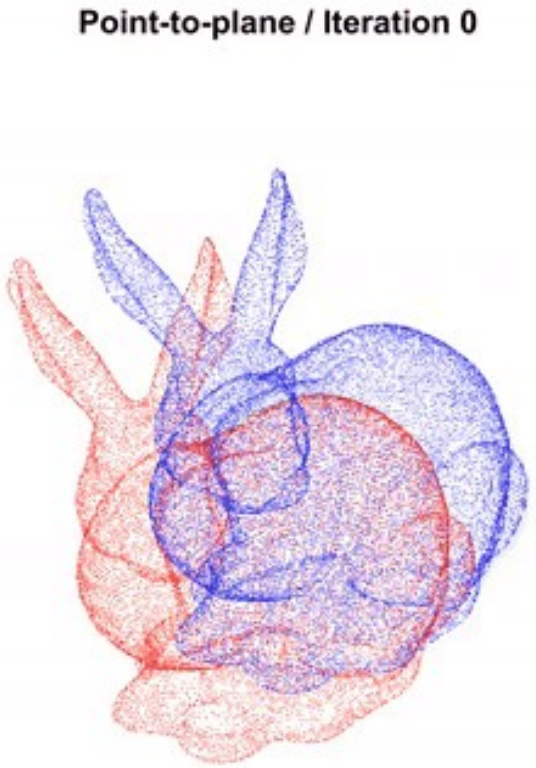
The **Iterative Closest Point** (ICP) algorithm rotates and translates the new scan until it best overlaps with the old scan.
The required rotation and translation to make them match is exactly how much the robot moved.

This provides highly accurate odometry without relying on wheel friction!

SLAM

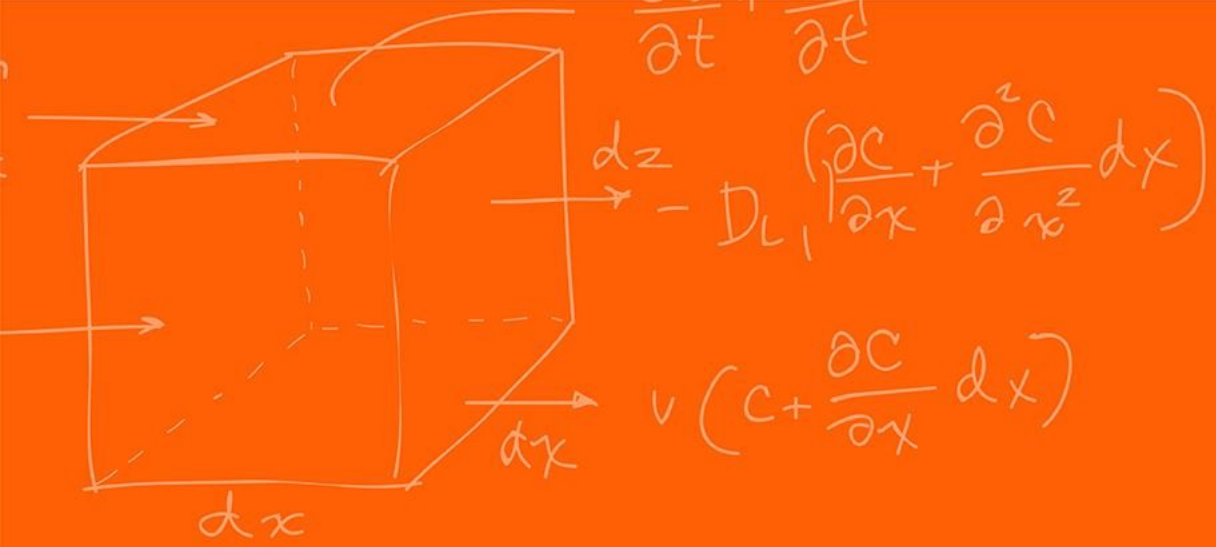




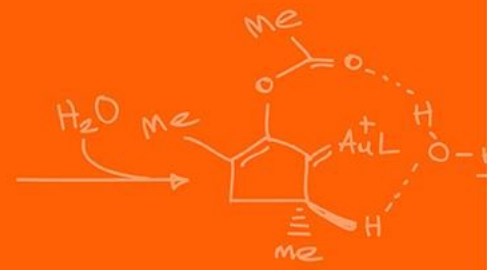


https://learnopencv.com/iterative-closest-point-icp-explained/

Questions?









The Grainger College of Engineering

UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

